TradeIO

Technical Specification Document

Trading Journal and Behavioral Analytics Platform

Senior Design

Assignment #9: Requirements Document and Technical Specification Document

Team Members

Ishaa Jain

Samarth Prajapati

October 30, 2026

Project Goal

TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

Contents

1 Version and Change Log **2**

2 Architecture Summary **2**

3 Component Interfaces and Data Formats **2**

3.1 Interface Contracts 2

3.2 API Endpoint Contract 3

3.3 Persistent Data and Processing Rules 4

4 Technology Stack **4**

5 Known Constraints and Design Consequences **5**

6 Requirements Traceability Matrix **5**

7 Advisor Review Record **6**

8 Controlled References **6**

1. Version and Change Log

Version	Date	Change	Authors
0.1	October 30, 2026	Team-approved technical specification aligning D0–D2, interfaces, technology choices, constraints, and Requirements Document v0.1.	Ishaa Jain and Samarth Prajapati

2. Architecture Summary

TradeIO is specified as a web-based client–server application with a layered server architecture. The user-facing component (C1) provides forms for trade entry, journals, risk settings, goals, and account preferences, and presents trading and behavioral summaries. Requests reach the Application/API Layer (C2), which validates the authenticated session, verifies account ownership, checks fields and business rules, and coordinates the operation. Valid trade and journal submissions pass to Trade and Journal Processing (C3) for normalization, then to Data Storage (C5) for transactionally consistent persistence.

Risk and Behavior Analysis (C4) retrieves account-scoped records through C5 and performs deterministic calculations. D1 Algorithm A governs time-zone-aware risk evaluation and limit flags; Algorithm B derives behavioral summaries from available trade and reflection data. The API returns metrics and warnings to C1, distinguishing an empty result from unavailable or outdated analysis. Critical calculations will not depend on generated AI content.

The planned deployment places the React/Next.js interface and TypeScript API routes on Vercel, with PostgreSQL hosted by Neon. External identity management (E1) supplies authenticated subject claims, but C2 remains responsible for object-level authorization. Optional AI insight (E2) receives reduced period summaries, while an external notification provider (E3) handles account-authorized alerts. These services will be isolated from core trade transactions so their failure cannot reverse a valid database commit. Broker connectivity is an exploratory integration rather than a release prerequisite; manual trade entry remains available.

Design references: TradeIO Design Diagrams; D0, *High-Level System Design*, Figures 1–3 and interfaces I1–I10; D1, *Detailed Design*, Figure 1 and Algorithms A–B; D2, *Detailed Design*, Figures 1–2 and error-case register. The diagrams form the referenced architectural baseline.

3. Component Interfaces and Data Formats

3.1 Interface Contracts

All external application API traffic will use HTTPS with UTF-8 JSON request and response bodies. Decimal monetary quantities will be represented as strings in the API and exact decimal types in storage. Time instants will be persisted in UTC; local trading periods will be determined from validated IANA time-zone settings. The server will derive the user identity from a verified session rather than trusting a user ID supplied in a request.

Boundary	Inputs	Outputs / format	Protocol	Error behavior
I1: C1 / C2	Trade details, account, direction, positive quantity and prices; JSON	Trade identifier and save status; JSON	HTTPS	Invalid fields: 400; missing session: 401; unauthorized account: 403; no write on failure

Boundary	Inputs	Outputs / format	Protocol	Error behavior
I2: C2 / C3	Validated trade or journal object; typed JSON	Normalized record and warnings; typed object	Internal service call	Invalid quantity, date or instrument: structured validation rejection
I3: C3 / C5	Normalized record, account scope, unique request key; relational data	Stored record ID and timestamp	Database transaction	Rollback on failure; duplicate key reconciled; unavailable storage: 503
I4: C5 / C4	Owned trades, journals, settings and requested period; relational records	Analysis dataset and period record	Indexed database query	Empty dataset reported explicitly; query failure does not publish a current result
I5: C4 / C2	Deterministic risk and psychology calculations	Metrics, flags and summary; JSON	Internal service call	Invalid range: 400; unavailable analysis identified as such
I6: C1/C2 / E1	Session credentials or validated token claims	Authenticated subject and session validity	HTTPS / identity SDK	Expired session: 401; forbidden account: 403; provider outage: 503
I7: C2 / C1	Selected account and reporting period; JSON	Dashboard metrics, recent trades, goals and freshness state; JSON	HTTPS	Explicit loading, empty, stale and error states; retry available
I8: C4 / E2	Minimized time-period statistics; JSON	Optional coaching explanation and suggestions; JSON	HTTPS API	Timeout, malformed response or rate limit: omit insight; retain deterministic summary
I9: C1 / C2	Currency, time zone, risk and notification preferences; JSON	Save confirmation and effective timestamp; JSON	HTTPS	Invalid values: 400; conflicting update: 409; previous valid settings retained
I10: C2 / E3	Authorized event, destination channel and redacted message; JSON	Delivery state and optional provider reference; JSON	HTTPS API	Provider failure logged and retried within limits; in-app event preserved

3.2 API Endpoint Contract

The initial REST contract follows D1's `/api/v1` convention. D2 defines further candidate endpoints; those extensions will be finalized when the corresponding requirements and policies are approved. An authenticated session and server-side ownership check are required for protected resources. Errors will use the standardized D2 response envelope.

Method	Path	Request	Success	Errors	Basis
POST	/api/v1/trades	Account UUID, ISO-8601 time, long/short, positive quantity and price	201 tradeId/status	400, 401, 403, 409, 500/503	D1
GET	/api/v1/trades	Account UUID, date range, page size 1–100, optional cursor	200 items/nextCursor	400, 401, 403, 404, 500	D1
GET	/api/v1/accounts/{id}/analysis	Account UUID and optional date range	200 metrics/flags/behavior summary	400, 401, 403, 404, 500	D1
PUT	/api/v1/accounts/{id}/risk-settings	0–100 percent limits, positive cap if enabled	200 saved/effectiveAt	400, 401, 403, 404, 409, 500	D1
POST	/api/v1/journal	Date, reflection, optional owned trade ID and scores	201 journalId/createdAt	400, 401, 403, 500	D1
GET	/api/v1/accounts/{id}/dashboard	Account ID and reporting period	200 dashboard plus freshness state	400, 401, 403, 503	D2 proposed
PUT	/api/v1/profile/preferences	IANA time zone and ISO currency	200 saved/effectiveAt	400, 401, 409	D2 proposed
POST	/api/v1/accounts/{id}/block	Partner authorization and 10 min–24 h duration	Block ID and expiry	400, 401, 403, 409	D2 proposed
GET	/api/v1/notifications	Authenticated notification query	Authorized alert collection	401, 403, 503	D2 proposed

Additional interfaces: REQ-10 requires goal creation and updates. The planned C1–C2–C5 boundary will accept an authenticated account or user scope, a weekly or monthly period, target value, and goal

status, and return a goal identifier or updated record. Invalid targets will be rejected without changing stored data, and ownership will be checked before any write. The exact HTTP method, route, and response schema remain to be specified before implementation. Journal export and broker synchronization also require separate contracts; broker synchronization remains conditional on scope approval.

Error format: An unsuccessful request will carry an HTTP status and a structured error containing `code`, `message`, `requestId`, `recoverable`, and optional `fieldErrors`. Responses will not reveal database internals, access tokens, or stack traces. Standard classes include 400 validation, 401 authentication, 403 authorization, 404 not found, 409 conflict, 429 throttling, 502 upstream response failure, 503 unavailable dependency, and 500 unexpected server error.

3.3 Persistent Data and Processing Rules

The D1 relational model defines `USER`, `TRADING_ACCOUNT`, `TRADE`, `JOURNAL_ENTRY`, `RISK_SETTINGS`, `GOAL`, and `ANALYSIS_RECORD`. Foreign keys enforce ownership relationships and valid references. Composite indexes support account and time-range trade retrieval, closed-trade calculations, dated journals, and stored period analytics. D2 identifies possible additional entities for fills, accountability, provider connections, audits, notifications, and insight jobs; these extensions are conditional rather than elements of an approved D1 schema.

Trade mutations will be validated and committed atomically. Idempotency identifiers will prevent duplicate submissions and imports. Concurrent limit checks will be reevaluated in an account lock or serializable transaction. Realized P&L will exclude open positions; missing instrument conversion factors will produce an unverified amount rather than an invented calculation. If a late import changes historical inputs, affected analytical summaries will be recomputed.

D1 Algorithm A derives local daily and weekly boundaries using IANA time zones, converts them to UTC, and compares realized results with configured limits using inclusive thresholds. Algorithm B computes reproducible behavior aggregates; absent scores remain missing, and periods without closed trades must not produce division-by-zero or fabricated percentages. Optional AI may explain these results but will not replace either algorithm.

4. Technology Stack

Layer	**Technology choice**	**Technical rationale and trace**
User interface	React, Next.js, Tailwind CSS	Supports structured forms, dashboards and accessible controls (REQ-03, REQ-09, NFR-07).
Server and APIs	Next.js API routes; TypeScript	Retains a single deployable application and typed validation (REQ-01, REQ-03, CON-01).
Persistent database	PostgreSQL on Neon	Supports account relationships, transaction integrity and indexed queries (REQ-04, NFR-04, CON-04).
Data access	Drizzle ORM	Supports typed relational records and exact decimal handling (NFR-06).
Identity	Clerk with server-side ownership validation	Uses a dedicated identity service while enforcing account isolation within TradeIO (REQ-01, NFR-02).
Deployment	Vercel and Neon	Minimizes infrastructure administration within a student budget (CON-01).
Time handling	JavaScript Intl and IANA identifiers	Preserves UTC instants and user-local reporting periods (REQ-14).
Optional AI	Provider-neutral API adapter	Keeps coaching supplementary and deterministic analytics independent (REQ-11, NFR-05, CON-02).

Layer	Technology choice	Technical rationale and trace
Optional messaging	External notification adapter	Separates message delivery from persistent trading records (REQ-12, NFR-09).
Private files	Protected object storage; provider to confirm	Enables restricted screenshot access within cost and privacy limits (CON-01, CON-03, CON-04).

5. Known Constraints and Design Consequences

ID	Inherited constraint	Architectural consequence
CON-01	Economic: student-accessible hosting, limited expenditure on AI and storage.	Favors a web-first deployment, manual trade entry and rate-controlled optional services; excludes native-mobile-first and paid-data-dependent designs.
CON-02	Ethical: explanatory coaching, no signals, order execution or promised returns.	Requires deterministic numerical results, advisory wording, and sample-size limitations for behavioral interpretations.
CON-03	Legal and privacy: protected financial records and screenshots; original MVP excludes direct broker connection.	Requires user-scoped records and protected attachments. Exploring one authorized broker connection remains a separately approved stretch decision.
CON-04	Security: secure authentication, authorization and protected storage.	Requires verified sessions, resource-level ownership checks, minimized outbound data and restricted credentials/media.

6. Requirements Traceability Matrix

The following matrix traces Requirements Document version 0.1 to the D0, D1 and D2 design specifications. *Designed* means that design information has been specified, not that software has been implemented. *Planned* indicates an intended but incomplete or conditional design. Test-case IDs are reserved for the Week 10 validation plan and do not represent executed tests.

Req ID	Description	Design Component (D0/D1/D2)	Status	Test Case
REQ-01	Authenticated account access	D0 C2/E1 I6; D2 ownership guard	Designed	TC-001
REQ-02	Trader-owned accounts	D1 TRADING_ACCOUNT; D0 C5	Designed	TC-002
REQ-03	Validated manual trade entry	D1 POST trades; D2 C3 and error path	Designed	TC-003
REQ-04	Filtered trade history	D1 TRADE index and GET trades	Designed	TC-004
REQ-05	Private journal and psychology entries	D1 JOURNAL_ENTRY; Algorithm B	Designed	TC-005
REQ-06	Deterministic performance	D1 Algorithm A/B; D0 C4	Designed	TC-006
REQ-07	Risk settings validation	D1 RISK_SETTINGS and PUT API	Designed	TC-007
REQ-08	Inclusive risk and discipline flags	D1 Algorithm A; D2 EC05/EC06	Designed	TC-008
REQ-09	Dashboard and freshness states	D0 C1/I7; D2 dashboard contract	Designed	TC-009
REQ-10	Weekly/monthly goals	D1 GOAL; D2 F10; write API open	Planned	TC-010
REQ-11	Optional AI explanations	D0 E2/I8; D2 AI adapter	Designed	TC-011
REQ-12	Event-based accountability alerts	D0 E3/I10; D2 notification outbox	Designed	TC-012
REQ-13	Per-account currency	D1 TRADING_ACCOUNT; D0 I9	Designed	TC-013
REQ-14	User-local time boundaries	D1 USER.time_zone; Algorithm A	Designed	TC-014
REQ-15	Optional broker import	D2 proposed broker adapter; scope open	Planned	TC-015
REQ-16	Authorized temporary cooldown	D2 block state diagram; scope open	Planned	TC-016
REQ-17	Cap-reached event, warning and partner alert	D2 F18/RiskGate; enforcement policy open	Planned	TC-017
REQ-18	Optional multi-fill trades	D2 proposed TRADE_FILL; D1 gap	Planned	TC-018

Req ID	Description	Design Component (D0/D1/D2)	Status	Test Case
REQ-19	Private data export	D0 C1/C3; Week 8 T11; API open	Planned	TC-019
NFR-01	Trade-history latency target	D1 indexed TRADE; D2 N09	Designed	TC-101
NFR-02	Cross-user data isolation	D0 C2/E1; D2 ownership guard	Designed	TC-102
NFR-03	Trade-entry usability	D2 C1 form flow	Designed	TC-103
NFR-04	Idempotent repeated submissions	D2 EC02/EC06/EC10; C5	Designed	TC-104
NFR-05	Core operation during AI outage	D1 Algorithm B; D0 I8	Designed	TC-105
NFR-06	Exact decimal precision	D1 decimal DTO; D2 C3	Designed	TC-106
NFR-07	Keyboard accessibility	D2 C1 accessibility rules	Designed	TC-107
NFR-08	Outbound sensitive-data minimization	D2 E2 redaction and audit	Designed	TC-108
NFR-09	Dependency failure recovery	D2 outbox and manual fallback	Designed	TC-109
CON-01	Low-cost web deployment	D0 client/server; D1 stack	Designed	TC-201
CON-02	Advisory AI limitations	D0 AI separation; D1 Algorithm B	Designed	TC-202
CON-03	Privacy and broker MVP boundary	D0 C2/C5; D2 broker gap	Designed	TC-203
CON-04	Secure identities and storage	D0 E1/I6; D2 auth glue	Designed	TC-204

7. Advisor Review Record

Earlier project consultation. The team discussed the TradeIO concept with Dr. William Hawkins III on August 28, 2026. During that discussion, the advisor suggested examining the feasibility of broker integration. The *TradeIO MVP Concept and Design Specification* sent on August 29, 2026 subsequently identified a single broker connection as a potential stretch goal. This recommendation did not establish broker integration as a required MVP feature.

Planned advisor review. The team plans to meet with Dr. Hawkins in the coming weeks to review the Requirements Document and Technical Specification. The team has approved the current requirements, priorities, and non-functional acceptance thresholds. The planned discussion will focus on remaining implementation policy and interface questions, including the broker release boundary, accountability cooldown and daily-cap behavior, multiple fills, and API contracts. No Assignment 9 advisor review or advisor-requested changes are claimed at this stage; the actual meeting date and outcomes will be recorded following the review.

Related peer feedback. D2 peer reviewer Harsh Parsana identified questions concerning risk-setting changes, expiration of temporary restrictions, and duplicate trade records. These matters remain visible in the corresponding error behavior and unresolved design decisions; peer feedback is not represented as faculty advisor approval.

8. Controlled References

Requirements Document, version 0.1; User Stories (September 28, 2026); D0 High-Level System Design; D1 Detailed Design; D2 Detailed Design; Week 3 Constraints Essay; Week 8 Implementation Planning.

Design Diagrams (D0, D1, and D2)